1장 깃과 버전 관리

이 장에서는 본격적으로 학습하기에 앞서 깃의 필요성과 특징을 간단히 알아보겠습니다. 깃의 유용성과 다양한 적용 분야를 발견할 수 있습니다.

1.1 버전 관리

1.2 버전 관리 시스템

1.3 깃

1.4 깃의 동작 한눈에 보기

1.5 정리

1.1 버전 관리

프로그래밍은 컴퓨터 언어로 글을 작성하는 창작 활동이라고 할 수 있습니다. 또 프로그래밍 개발 과정은 수많은 코드를 변경하고 테스트하는 것입니다. 지속적으로 변경되는 과정 속에서 코드는 잠시 불안정한 수정 상태와 안정된 상태를 반복합니다. 개발자는 안정된 상태의 코드와 불안정한 상태의 코드를 인지하고, 항상 안정된 상태를 유지하도록 노력해야 합니다.

1.1.1 버전이란?

버전이란 무엇일까요? 컴퓨터 프로그램을 사용하면서 버전(version)이라는 단어를 많이 들어 보았을 것입니다. 대부분은 윈도 10, 한글 9.7 등 프로그램 이름 뒤에 붙어 있는 숫자가 버전을 의미한다고 알고 있을 것입니다. 맞습니다. 이 숫자가 버전입니다. 사전을 찾아보면 이전의 것과 다른 또는 약간 비슷한 것으로 정의되어 있습니다.

버전은 이전과 약간씩 다른 변화들을 구분하는 표시입니다. 버전을 표시하는 데 숫자를 많이 사용하지만 꼭 숫자만 사용해야 하는 것은 아닙니다. 2016에디션, 2017에디션, 윈도 XP, macOS X 엘카피탠처럼 연도나 다른 기호를 사용하기도 합니다.

그렇다면 서브버전은 무엇일까요? 보통 버전과 버전 사이에도 코드가 수없이 변화되며 이 중간에도 버전이 있습니다. 이처럼 버전과 버전 사이에 변화된 것을 서브버전이라고 합니다. 1.0 버전과 2.0 버전 사이에는 1.01, 1.02, 1.03처럼 수많은 서브버전이 있습니다.

이러한 버전의 숫자나 기호 역시 일련의 규칙들이 있습니다. 버전을 부여하려면 소스 코드를 구별할 수 있는 의미 있는 변화가 있어야 합니다. 개발 도중 임시로 작업한 것을 버전이라고 말하지는 않습니다.

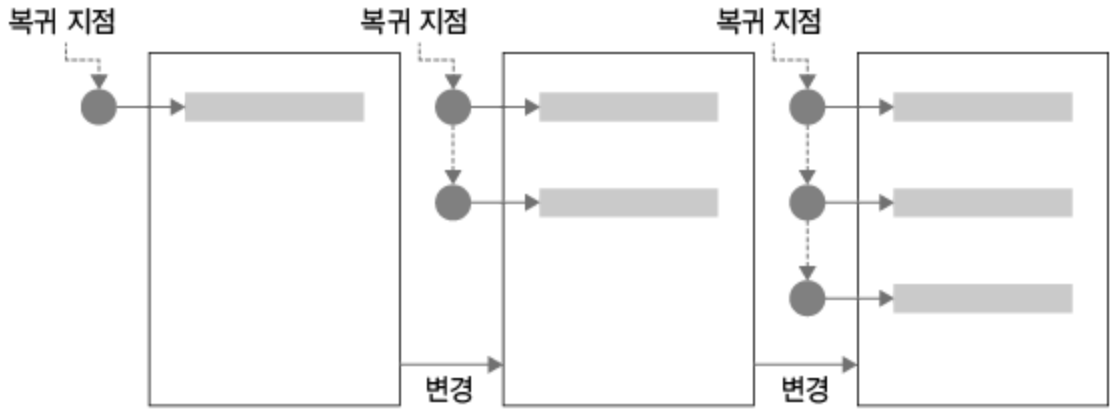
1.1.2 버전 관리는 왜 필요할까?

어릴 적부터 많이 듣던 말 중에 “공부에는 왕도가 없다.”가 있습니다. 마찬가지로 프로그램 코드도 한 번에 모두 완성되지 않습니다. 일반적으로 프로그램 코드는 한 단계씩 살을 붙여 가면서 기능들을 추가해 나갑니다. 오랜 시간 동안 수많은 작업을 개선하면서 데이터는 변하고 끊임없이 증가합니다. 이러한 변화와 작업을 진행하면서 하나의 완성된 프로그램을 만드는 것입니다.

개발 도중에는 많은 기능이 추가되고, 수많은 코드가 변경됩니다. 변경되는 동안 코드들은 잠시 불안정한 상태가 되고, 이후 정상적인 테스트와 동작을 확인하고 나면 다시 안정된 상태의 코드가 됩니다.

숙련된 개발자라면 작업 과정에서 코드들을 안정되게 유지해야 한다는 것을 잘 압니다. 개발 또는 테스트하는 과정에서 불안정한 코드가 있다면 계속 이어서 작업하기 불안할 것입니다. 때에 따라 더 이상 작업하기 어려울 수도 있습니다. 이때는 이전 상태로 돌아가 다시 시작할 수 있는 코드의 복귀(포인트) 지점이 필요합니다.

▼ 그림 1-1 코드 복귀 지점



코드 복귀 지점은 반드시 안정된 코드 상태를 기준으로 설정해야 합니다. 이렇게 복귀 지점을 기록해 두면 좀 더 자유롭고 안정적으로 개발할 수 있습니다.

1.2 버전 관리 시스템

우리는 컴퓨터를 사용하면서 이미 다양한 방법으로 버전을 기록하고 있습니다. 예를 들어 워드프로세서로 보고서를 작성할 때 갑자기 컴퓨터 전원이 꺼져서 작업한 내용이 사라진 적이 있을 것입니다. 이러한 경험을 하게 되면 자신도 모르게 작업 상태를 수시로 저장하는 습관이 생깁니다. 이처럼 작업 상태를 저장하는 것 또한 변화의 기록입니다.

일반적으로 우리는 한 파일에 계속 덮어 쓰면서 저장합니다. 이때 변화되는 버전을 표시하지 않았기 때문에 이전 상태로는 돌아가지 못합니다. 변화가 있는 문서 상태를 구분하고 싶을 때는 보통 '파일이름_1, 파일이름_2'처럼 파일 이름을 다르게 저장하곤 합니다. 이 다른 이름으로 저장하는 방식은 한 문서를 여러 파일로 계속 저장하는 방식이라, 시간 차이를 두고 저장하다 보면 이름 규칙을 잊곤 합니다. 다음과 같이 통일성 없이 파일 이름을 만들어 저장했다면 버전을 알아보기 힘듭니다.

▼ 그림 1-2 새 파일로 계속 저장



그렇다면 파일 버전을 좀 더 편리하고 정확하게 관리할 수는 없을까요? 이때 사용하는 것이 버전 관리 소프트웨어입니다. 특히 프로그래밍 코드는 일반 문서와 달리 복잡하고 많이 변경되므로 버전 관리 소프트웨어를 사용하는 것이 좋습니다.

1.2.1 버전 관리 소프트웨어

코드와 콘텐츠의 변화를 관리하고 추적하는 소프트웨어를 버전 관리 시스템(VCS, Version Control System)이라고 합니다. 최초의 버전 관리 시스템은 유닉스 환경에서 사용 가능한 SCCS(Source Code Control System)입니다. SCCS는 1970년대 마크 로치킨드(Marc J. Rochkind)가 개발했으며, 이후 다양한 VCS 프로그램이 등장합니다. VCS에서 버전 파일들을 관리하고 저장하는 공간을 저장소(repository)(리포지터리)라고 합니다.

현재 사용하는 VCS는 크게 집중형과 분산형, 두 종류로 구분할 수 있습니다.

집중형

집중형 시스템은 말 그대로 모든 소스 코드가 한곳에 집중되어 있는 형태입니다. 하나의 메인 중앙 서버에서 개발 구성원의 모든 소스 코드를 통합적으로 관리합니다. 클라이언트-서버 모델이라고도 합니다.

- 장점: 저장소 하나를 중심으로 관리하기 때문에 시스템을 운영하기 수월합니다.
- 단점: 중앙 저장 공간인 서버에 문제가 생기면, 소스 코드가 있는 메인 저장소에 모든 개발자가 접근할 수 없는 심각한 상황이 발생할 수 있습니다. 또 동시에 여러 개발자가 접근하면 충돌이 발생하기에 코드 수정을 안정적으로 할 수 있게 잠금 모델을 적용합니다. 파일을 변경하려면 개발자들은 순서대로 대기하고 있어야 합니다.

Note 잠금 모델

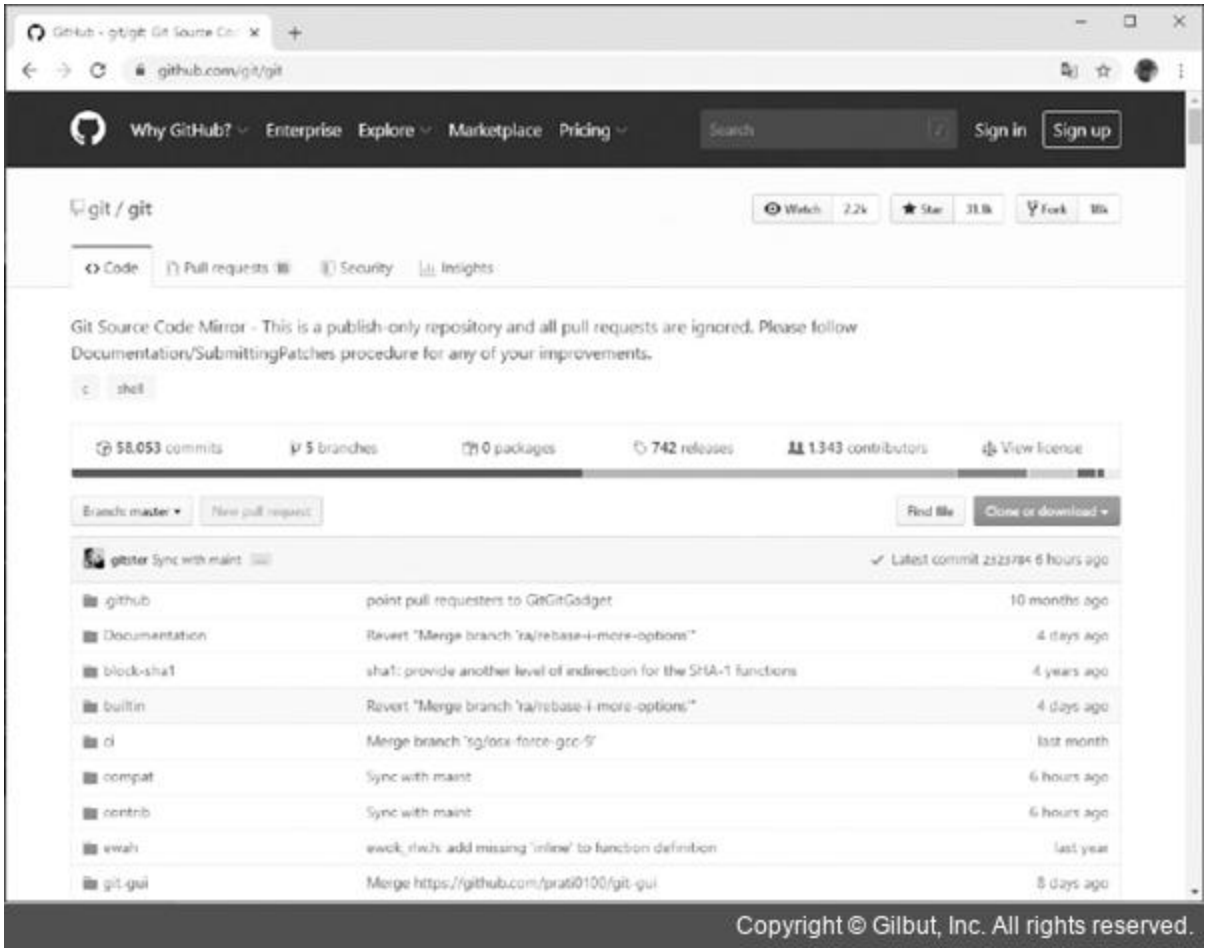
보통 다수의 클라이언트가 서버에 동시 접속하곤 합니다. 파일 하나를 다수의 사용자가 동시에 접근하여 수정하려고 한다면 충돌이 발생합니다. 누군가 먼저 파일을 수정하고 있다면 다른 사용자는 수정을 못하도록 잠금을 설정하여 방지할 수 있습니다.

1.3 깃

깃은 2005년에 리눅스 개발자인 리누스 베네딕트 토르발스(Linus Benedict Torvalds)가 개발했습니다.1 깃의 모든 소스는 깃허브2에 공개되어 있으며, 깃허브에서 git으로 검색하면 됩니다.

https://github.com/git/git

▼ 그림 1-3 깃 소스



1.3.1 백업 기능

분산형 깃은 자신의 로컬 컴퓨터에서 독립적으로 소스의 버전 관리를 할 수 있습니다. 독립적이란 것은 로컬 컴퓨터에서 자체적으로 버전을 기록하고 관리할 수 있는 시스템을 의미합니다. 혼자 진행하는 프로젝트에서는 이것만으로도 충분합니다. 하지만 컴퓨터에 문제가 생긴다면 지금까지 개발한 모든 소스를 잃을 수 있습니다. 이에 대비하여 별도의 외부 저장 장치에 데이터를 백업하곤 합니다. 하지만 외부 저장 장치에 백업하는 코드는 단순히 파일을 복사하는 것으로 기존 소스 코드와는 동기화하지 않은 별개의 파일입니다. 프로젝트 개발자라면 코드를 좀 더 안전하게 유지하고 백업하는 데 항상 신경 써야 합니다.

이때 깃을 사용하면 편리합니다. 깃을 사용하면 코드를 원격 저장소에 저장할 수 있습니다. 또 로컬 컴퓨터의 저장소를 동기화하여 원격 저장소에 백업합니다. 그리고 사무실, 집 등 여러 공간에서 원격 저장소에 저장된 내용을 다시 내려받아 프로젝트 개발을 이어서 할 수 있습니다.

1.3.2 협업 개발

깃은 다수의 개발자와 코드를 공유하고 협업할 때 매우 유용합니다.

코드 공유

예전에는 팀 내에서 코드를 공유하려고 외부 저장 장치를 이용하곤 했습니다. 이는 매우 번거로운 작업입니다. 깃을 사용하면 네트워크를 통해 코드를 좀 더 쉽게 공유할 수 있습니다. 심지어 인터넷이 연결되지 않은 상태에서도 코드 이력을 관리하고, 다른 개발자와 공유하여 협업할 수 있습니다.

책임과 기록

깃은 변경된 모든 이력을 저장합니다. 누가 언제 어떤 파일을 수정했는지 기록하기 때문에 코드를 좀 더 책임감 있게 작성하고 유지할 수 있습니다. 깃은 커밋(commit)5을 거쳐 모든 코드의 수정 이력을 기록합니다. 따라서 깃의 커밋은 신중하게 작업해야 합니다. 커밋으로 저장된 원본 객체는 수정할 수 없습니다.

원격 공유

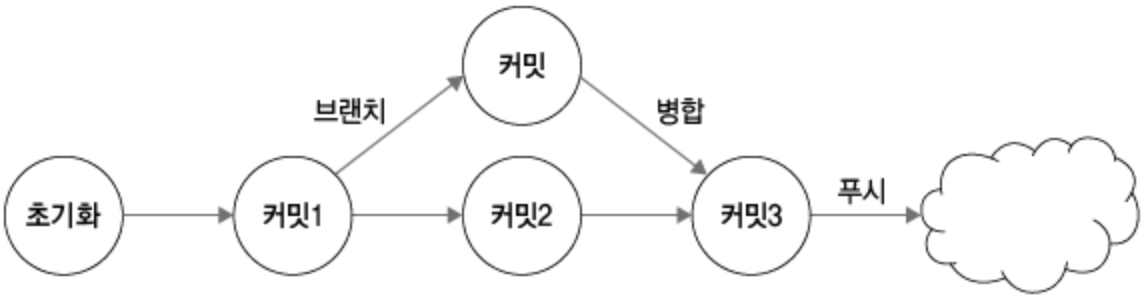
분산된 여러 저장소 간에 정보를 주고받으려면 중앙 서버가 필요합니다. 깃에서는 원격 저장소가 중앙 서버 역할을 합니다. 자신의 코드 저장소를 원격 서버에 푸시(push)6하여 저장(동기화)합니다. 또 다른 개발자의 소스를 원격 서버에서 풀(pull)7 또는 페치(fetch)8하여 언제든지 내려받을 수 있습니다.

깃을 사용하면 개발 구성원 간에 소스 코드를 쉽게 주고받을 수 있습니다. 원격 저장소로 전송된 코드는 여러 개발자와 소스 코드를 공유합니다. 협업하여 코드를 개발할 때 공유 기능은 매우 중요합니다.

1.4 깃의 동작 한눈에 보기

앞으로 깃의 다양한 동작을 살펴볼 것입니다. 깃은 크게 다섯 단계로 진행됩니다.

▼ 그림 1-4 깃의 다섯 단계



- 초기화: 폴더를 깃 저장소로 변경합니다.
- 커밋: 변경된 코드의 이력을 기록합니다.
- 브랜치: 분리 격리된 코드 이력을 기록합니다.
- 병합: 기존 이력과 분리된 이력을 통합합니다.
- 푸시: 로컬 저장소의 이력을 서버로 전송 및 공유합니다.

다음 장부터 각 단계를 순차적으로 학습할 것입니다.

1.5 정리

깃은 다양한 알고리즘과 기술로 무장하고 있습니다. 또 많은 프로젝트를 깃으로 관리하면서 안정성과 가치를 인정받고 있습니다. 깃을 잘 활용하려면 깃의 개념을 이해하고 많이 사용해 보아야 합니다. 앞으로 장별로 제공하는 실습을 따라 하면서 여러 가지 깃의 기능을 학습할 것입니다. 깃을 잘 활용하여 자신의 개발 능력을 한 단계 더 높일 수 있는 계기가 되었으면 합니다.

1 영국 영어에서 Git은 '바보, 가치 없는 사람'이라는 반어적 표현입니다.

2 깃허브는 나중에 배웁니다. 여기서는 깃 소스가 모두 공개되어 있다는 점만 알고 넘어갑니다.

3 원격 서버에서 관리하는 저장소(외부 저장소)입니다. 자신의 원격 저장소 외에 타인의 저장소도 포함하며, 여러 명과 공유할 수 있습니다.

4 간단히 두 소스 코드를 합치는 것입니다. 자세한 내용은 8장에서 설명합니다.

5 파일 및 폴더의 추가·변경 사항들을 기록하는 것을 의미합니다. 이 커밋을 기준으로 코드를 관리합니다.

6 자신의 로컬 저장소(개인 PC의 저장소)에서 변경된 파일을 원격 저장소로 업로드하는 것입니다.

7 원격 저장소의 변경 사항을 자신의 로컬 저장소에 적용하는 것입니다.

8 원격 저장소의 변경 사항을 내려받는 것으로, 적용(병합)은 하지 않습니다.

9 메인 개발 라인에서 분기된 하나의 갈래를 의미합니다. 새로운 기능을 안전하게 개발하고 싶을 때 추가로 메인 개발 라인에서 분기된 새 개발 라인을 만들어 개발하고, 개발이 완료되면 다시 메인 개발 라인에 병합합니다. 6장에서 자세히 설명합니다.

10 다른 사람의 원격 저장소를 가져와(복사하여) 자신의 원격 저장소를 만드는 것을 의미합니다.

11 자신의 로컬 저장소에서 작업한 후 이 작업을 원본 저장소에 반영해 달라고 요청하는 것을 의미합니다.